
Software Requirements Specification

for

Analytics BI,Dashboard creation and Visualization

Group Members:

Vuppula Hemanth Reddy	AI23BTECH11033
Sampadram	AI23BTECH11025
Sukesh	AI23BTECH11024
Rinku Kumari	ES23BTECH11032
Sai Deepak Sana	CS23BTECH11055
Vishal Varukolu	CS23BTECH11064

Table of Contents

Revision History	ii
1. Introduction	1
1.1 Purpose	1
1.2 Problem statement	1
1.3 Intended Audience	1
2. Overall Description	2
2.1 Product Perspective	2
2.2 Product Features	2
2.3 User Classes and Characteristics	2
2.4 Assumptions and Dependencies	3
3. System Features	3
3.1 Data Ingestion	5
3.2 Data Cleaning and Relationship mapping	6
3.3 Query Builder	7
3.4 SQL Query Interface	8
3.5 Visualization	9
3.6 Dashboard customization	10
3.7 Collaboration	11
3.8 Alert system	12
3.9 Export System	13
3.10 Chatbot assistant	14
3.11 Predictive Analytics	15
3.12 AI SQL Assistant	16
3.13 Other AI features	17
3.14 Version control and history	18
4. Sprint 1 Scope	19
4.1 Sprint goal	19
4.2 Deliverables	19
4.3 Sprint 1 work Distribution	19
4.4 Technology stack (Sprint 1)	20
4.5 Data Flow	20

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

The project “**Analytics BI, Dashboard Creation and Visualization**” is designed to enable users to generate analytical reports from ingested data and transform them into meaningful charts and visualizations. The platform provides an interactive dashboard environment where users can explore insights dynamically. These dashboards can be exported in various file formats or embedded into other applications using iframe integration, ensuring flexible sharing and accessibility of insights.

This document is intended to clearly define the features and requirements of this project, serving as a comprehensive guide for the development team during design and implementation, and as a software validation reference for prospective clients to ensure the system meets agreed business objectives and expectations.

1.2 Problem statement

Build a platform that enables users to ingest data, build dashboards, run queries, and visualize KPIs. It should offer drag-and-drop visualization builders, reusable widgets, scheduled reports, and supports data transformations and joins.

Objective: Empower stakeholders to create meaningful dashboards and derive insights from raw data without heavy engineering.

Scope: ETL-like ingestion, simple SQL/visual query builder, charting library, shareable dashboards, and scheduled exports.

1.3 Intended Audience

Businesses, researchers, educators, analysts, and individual users.

2. Overall Description

This section provides a comprehensive overview of the Analytics BI, Dashboard Creation, and Visualization Platform. It describes the system's operating environment, the expected level of technical knowledge from users, general constraints affecting system design and implementation, and the key assumptions and dependencies considered during development.

2.1 Product Perspective

The **Analytics BI, Dashboard Creation, and Visualization Platform** is a generalized data analytics solution designed for a wide range of users, including businesses, researchers, educators, analysts, and individual users. The platform enables users to ingest data from multiple sources, perform transformations and joins, run queries through visual or SQL-based builders, and create interactive dashboards to visualize KPIs and insights.

It provides drag-and-drop visualization tools, reusable widgets, and configurable charts to make data exploration simple and intuitive, regardless of domain. The system also supports scheduled reports, threshold-based alerts, and export or embedding options, allowing users to share insights easily across different platforms. The goal is to empower users from any field to analyze and present data effectively without requiring advanced technical expertise.

The product is designed as a stand-alone platform that does not depend on external software for its core functionality. It will run on both UNIX/Linux and Windows-based systems. The system is intended to be user-friendly, easy to learn, reliable, and stable, ensuring efficient performance for the defined problem statement.

2.2 Product Features

The platform provides the following major features:

1. Ingest data from multiple sources, perform transformations and joins, and run queries through visual or SQL-based builders.
2. Create interactive dashboards to visualize KPIs and insights using drag-and-drop visualization tools, reusable widgets, and configurable charts.
3. Support for scheduled reports, threshold-based alerts, and export or embedding options to easily share insights across different platforms.

2.3 User Classes and Characteristics

1. **Non-Technical Users (Viewers/General users):** Individuals with basic computer literacy who require intuitive, drag-and-drop tools for general data exploration.
2. **Technical Users (Editors/Analysts):** Data professionals familiar with SQL and data workflows who build advanced charts and complex analytical queries.

3. **System Administrators (Admins):** Highly privileged technical users responsible for platform governance, managing schema registries, and maintaining data pipelines.
4. **System (Automated):** The backend actor responsible for real-time stream processing, automated alerts, and integrity verification.

2.4 Assumptions and Dependencies

1. Assuming that Authentication is done and we receive a Json file that contains user information such as name,role etc

3. System Features

This section outlines the detailed requirements the software must fulfill. Each requirement is assigned a priority level to define its necessity for the project's completion:

- **High (Must-Have):** These are critical requirements that are essential for the software to function. The project cannot be considered successful without these features.
- **Medium (Recommended):** These are important features that should be included to provide significant value and improve the user experience.
- **Low (Optional):** These are enhancements that will be implemented if time permits.

High	Medium	Low
Data Ingestion	Query Interface	Chatbot assistant
Data cleaning and relationship mapping	Collaboration	Predictive analytics
Query builder	Alert system	AI Query assistance
Visualization	Export system	Version control and history
Dashboard customization		Other AI features

3.1 Data Ingestion

3.1.1 Description and Priority

Priority: High.

The system must handle the uploading of various types of data, such as Excel files and csv files, into the platform. It must securely manage data loading and handle structural complications, specifically giving the user control over how new data interacts with existing records.

3.1.2 Stimulus/Response Sequences

- The user uploads a data file (e.g., Excel) to the platform.
- The system prompts the user to define the ingestion behavior.
- The user selects an ingestion rule.
- The system processes the file according to the selected rule and stores the data.

3.1.3 Functional Requirements

- **REQ-1.1:** The system shall allow users to easily upload standard data files, such as Excel and CSV spreadsheets, Database files etc, into the platform.
- **REQ-1.2:** The system shall use a simple step-by-step "wizard" that asks users whether they want to create a brand new dataset, replace an old one, or just add the new data to the end of an existing file.
- **REQ-1.3:** The system shall also be able to receive continuous, real-time data streams (like live feeds) without interrupting normal operations.
- **REQ-1.4:** The system shall automatically scan all incoming data to verify its format and ensure it meets the system's quality standards.
- **REQ-1.5:** The system shall catch any corrupted data and move it to a separate "Quarantine" folder so it doesn't break the rest of the ingestion process.

3.2 Data Cleaning (DTS) and Relationship Mapping

3.2.1 Description and Priority

Priority: High.

Immediately following data ingestion, the system automatically runs Data Transformation Services (DTS) to standardize the data and prepare it for meaningful analysis. This stage serves two purposes: first, it automatically fixes data errors; second, it acts as a "relationship maker" by analyzing the dataset to identify and classify fields as either Dimensions (how the data is categorized, like 'Region' or 'Date') or Measures (the numerical values, like 'Revenue' or 'Units Sold'). This automation ensures that the subsequent visualization stage can generate accurate charts without manual setup.

3.2.2 Stimulus/Response Sequences

- Data is successfully ingested into the platform's raw storage.
- The system automatically triggers the DTS pipeline to cleanse the data and identify data types.
- The system identifies the schema and field types.
- The system automatically maps each field to a "Dimension" or "Measure" category and establishes the relationships needed for charting.
- The system encounters unfixable malformed data.
- The system routes these specific records to the Dead Letter Queue (DLQ) and notifies the administrator while allowing the valid, mapped data to proceed.

3.2.3 Functional Requirements

- **REQ-2.1:** The system shall automatically clean and format ingested datasets without requiring any manual work from the user.
- **REQ-2.2:** The system shall function as a "relationship maker" by automatically classifying every data field as either a **Dimension** or a **Measure**.
- **REQ-2.3:** The system shall automatically apply fix-it rules, such as replacing missing values (nulls) with zeros, to ensure data is ready for analysis.
- **REQ-2.4:** The system shall automatically alert administrators if it finds "unfixable" data that needs manual review in the Dead Letter Queue (DLQ).
- **REQ-2.5:** The system shall automatically link related data tables together so that the Visualization wizard has all the information it needs for complex charts.

3.3 Query Builder (The Data Wizard)

3.3.1 Description and Priority

Priority: High.

The Query Builder is a user-friendly interface that allows users to select exactly what data they want to analyze without writing any SQL code. It uses the "Dimensions" and "Measures" identified during the Data Cleaning stage to guide the user. This "Wizard" ensures that users only ask for data that makes sense, preventing errors before the visualization starts.

3.3.2 Stimulus/Response Sequences

- The user selects a dataset to explore.
- The system displays two columns: one for "Dimensions" (categories) and one for "Measures" (numbers).
- The user drags a dimension (e.g., 'Product Category') and a measure (e.g., 'Total Profit') into the query area.
- The system builds the query in the background and shows a "Data Preview" table.
- The user applies a filter (e.g., "Only show data for 2025").
- The system updates the preview to show only the filtered results.

3.3.3 Functional Requirements

- **REQ-3.1:** The system shall provide a **Visual Query Builder** that allows users to create data requests by dragging and dropping Dimensions and Measures.
- **REQ-3.2:** The system shall automatically generate the necessary backend code (SQL) to fetch data, so the user never has to write code manually.
- **REQ-3.3:** The system shall allow users to apply **Filters** (e.g., equals, greater than, contains) to narrow down the data being queried.
- **REQ-3.4:** The system shall provide a **Live Preview** table that shows the first 10 rows of the query results so the user can verify their data before moving to visualization.
- **REQ-3.5:** The system shall allow users to **Save Queries**, so they can use the same data selection for multiple different charts later.
- **REQ-3.6:** The system shall prevent "Invalid Queries" by only allowing users to perform math (Measures) on numerical data and grouping on categorical data (Dimensions).

3.4 SQL Query Interface

3.4.1 Description and Priority

Priority: Medium.

While the Query Builder provides a simple "wizard" for most users, the SQL Query Interface is designed for technical users and data analysts. It provides a professional code editor where users can write, test, and run custom SQL scripts directly against the cleaned datasets. This allows for complex data manipulation that might go beyond the standard drag-and-drop options.

3.4.2 Stimulus/Response Sequences

- The user switches from "Wizard Mode" to "SQL Mode."
- The system opens a code editor with syntax highlighting and a list of available table names.
- The user writes a **SELECT** statement and clicks "Run."
- The system executes the query and displays the results in a data grid below the editor.
- The user clicks "Create Visualization" from the SQL results.
- The system passes the resulting data table directly into the Visualization stage.

3.4.3 Functional Requirements

- **REQ-4.1:** The system shall provide a **SQL Editor** with syntax highlighting and auto-complete for table and column names.
- **REQ-4.2:** The system shall allow users to execute custom SQL queries and view the results in a searchable, paginated table.
- **REQ-4.3:** The system shall include a **Syntax Validator** that alerts the user to errors in their SQL code before they attempt to run it.
- **REQ-4.4:** The system shall allow users to save their custom SQL scripts as "Stored Queries" to be reused for future dashboards or reports.
- **REQ-4.5:** The system shall provide a **Query History** log, allowing users to see and reuse previously executed SQL statements.
- **REQ-4.6:** The system shall allow users to set "Read-Only" permissions for the SQL interface to ensure that no one accidentally deletes or modifies the underlying database.

3.5 Visualization

3.5.1 Description and Priority

Priority: High.

In this stage, the system takes the data that was automatically cleaned and categorized in the previous step and turns it into visual charts. To ensure the project remains stable and professional, the system uses established charting libraries (like Chart.js) rather than building graphics from scratch. The process is handled through a "wizard" that guides the user to select the right data for the right chart.

3.5.2 Stimulus/Response Sequences

- The user selects a prepared dataset and opens the visualization wizard.
- The system displays a list of available chart types (e.g., Bar, Line, 3D charts).
- The user picks a chart and assigns at least one "Dimension" and one "Measure".
- The system uses the charting library to instantly show a preview of the visualization.

3.5.3 Functional Requirements

- **REQ-5.1:** The system shall provide a step-by-step "wizard" interface that helps users create charts without needing technical coding skills.
- **REQ-5.2:** The system shall strictly require users to select at least one **Dimension** (the category) and one **Measure** (the number) for every chart to ensure the output is meaningful.
- **REQ-5.3:** The system shall utilize industry-standard libraries, such as **Chart.js** or **3D charting modules**, to generate and display all visual elements.
- **REQ-5.4:** The system shall allow users to customize the visual appearance, such as changing colors, labels, and legends, before saving the chart.
- **REQ-5.5:** The system shall allow users to save their finished visualizations so they can be reused later in different dashboards.

3.6 Dashboard Customization

3.6.1 Description and Priority

Priority: High.

This stage allows users to combine multiple visualizations into a single, interactive screen known as a dashboard. The goal is to provide a "personal workspace" where users can arrange their saved charts in a way that makes sense for their specific business needs. This section focuses on ease of use, using a grid-based layout so users can easily organize their insights.

3.6.2 Stimulus/Response Sequences

- The user creates a new dashboard or opens an existing draft in their personal workspace.
- The system provides a blank or existing grid where visualizations can be placed.
- The user selects saved charts and drags them onto the dashboard.
- The system snaps the charts into the grid and allows the user to resize them.
- The user applies a "Global Filter," such as a specific date range or region.
- The system automatically updates every chart on the dashboard to reflect that filter.

3.6.3 Functional Requirements

- **REQ-6.1:** The system shall allow users to create a private draft dashboard in their personal workspace to experiment with different chart layouts.
- **REQ-6.2:** The system shall provide a "drag-and-drop" interface that lets users arrange and resize visualizations on a flexible grid.
- **REQ-6.3:** The system shall allow users to add "Global Filters" to the dashboard, which change the data shown in all charts at once.
- **REQ-6.4:** The system shall allow users to add text annotations or comments to specific charts to explain what the data is showing.
- **REQ-6.5:** The system shall save the entire dashboard layout, including chart positions and filter settings, so it can be reopened exactly as it was left.

3.7 Collaboration

3.7.1 Description and Priority

Priority: Medium.

Once a user has created a meaningful dashboard, the system provides tools to share those insights with other team members. This section focuses on moving from a private "Personal Workspace" to a "Team Workspace," where multiple users can view and discuss the same data in real-time. This ensures that data-driven decisions can be made collectively across the organization.

3.7.2 Stimulus/Response Sequences

- A user completes a dashboard in their Personal Workspace and selects the "Share" option.
- The system prompts the user to select specific team members or a Team Workspace.
- The user grants "Viewer" or "Editor" permissions to a colleague.
- The system sends a notification to the colleague and adds the dashboard to their "Shared with Me" folder.

3.7.3 Functional Requirements

- **REQ-7.1:** The system shall allow users to move dashboards from a private Personal Workspace to a Shared Team Workspace for group access.
- **REQ-7.2:** The system shall support a "Comment" feature, allowing users to leave notes or feedback directly on specific dashboard components.
- **REQ-7.3:** The system shall allow the owner of a dashboard to set access levels, such as "View Only" or "Can Edit".
- **REQ-7.4:** The system shall provide a "Link Sharing" feature that generates a secure, unique URL for others to access the dashboard.
- **REQ-7.5:** The system shall keep a history of changes made to a shared dashboard, allowing users to see who edited a chart and when.

3.8 Alerts system

3.8.1 Description and Priority

Priority: Medium.

The system features an automated alerting system that monitors data for specific conditions. While the actual delivery of messages (email/SMS/Slack) is handled by an external notification team, our system is responsible for detecting when a threshold is crossed and communicating that event to the notification service.

3.8.2 Stimulus/Response Sequences

- A data measure checks pre-defined conditions, such as a sharp drop in sales.
- The system triggers an alert event and sends the relevant data to the notification team's service.
- The external notification team successfully delivers the message.
- The system receives and displays a "notification of success" from the alerting system for the user to see.

3.8.3 Functional Requirements

- **REQ-8.1:** The system shall allow users to set custom "Alert Rules" based on specific measures and dimensions.
- **REQ-8.2:** The system shall automatically monitor data streams and batch uploads to detect when these rules are violated.
- **REQ-8.3:** The system shall generate an alert payload (including chart data and text) and pass it to the external notification team's API.
- **REQ-8.4:** The system shall listen for and record a "success" confirmation from the external notification system once a message is sent.
- **REQ-8.5:** The system shall provide an internal log for users to track which alerts were triggered and whether they were successfully dispatched.

3.9 Export System

3.9.1 Description and Priority

Priority: High.

The Export System allows users to move their data and visual insights out of the platform for external use. This includes exporting files directly to a local device for offline work, as well as "Live Exporting" through embedding, which allows charts to appear on other company websites. This ensures that data remains portable and accessible across different tools and departments.

3.9.2 Stimulus/Response Sequences

- The user selects the "Export" icon on a specific chart or dashboard.
- The system displays a menu for the user to choose their preferred export format (e.g., CSV, PDF, or Iframe).
- The user chooses to export a dashboard as an Iframe.
- The system generates a unique HTML code snippet that can be used to embed the live chart elsewhere.
- The user sets up a recurring export for a monthly data summary.
- The system automatically processes the export and delivers the file to the chosen destination at the scheduled time.

3.9.3 Functional Requirements

- **REQ-9.1:** The system shall allow users to export raw data from any chart to their local device in CSV or Excel formats.
- **REQ-9.2:** The system shall allow users to export full dashboards as PDF or high-quality images for use in external reports or presentations.
- **REQ-9.3:** The system shall generate a secure Iframe code snippet as a "Live Export" option, allowing dashboards to be embedded and viewed inside other portals.
- **REQ-9.4:** The system shall ensure that any dashboard provided via an Iframe export stays "live," automatically reflecting updates made to the source data.
- **REQ-9.5:** The system shall allow users to Schedule Exports, so that data files are automatically generated and sent to cloud storage or notification system on a set recurring basis.
- **REQ-9.6:** The system shall maintain an Export Log that tracks the date, format, and user for every file exported from the system.

3.10 Chatbot Assistant

3.10.1 Description and Priority

Priority: Low.

The Chatbot acts as a natural language interface for the platform. Instead of navigating menus or setting up complex filters, users can simply "talk" to their data. It is designed to help non-technical users find insights quickly by asking questions in plain English.

3.10.2 Stimulus/Response Sequences

- The user types a question like "Which region had the highest growth last quarter?".
- The system translates the question into a data query, analyzes the dimensions, and replies with a text summary and the relevant chart.
- The user asks the chatbot to "Add this chart to my dashboard."
- The system automatically places the generated visualization into the user's personal workspace.

3.10.3 Functional Requirements

- **REQ-10.1:** The system shall provide a Chatbot interface that supports Natural Language Processing (NLP) to answer user questions about any ingested dataset.
- **REQ-10.2:** The system shall be able to identify Dimensions and Measures mentioned in a chat query to generate the correct visual response.
- **REQ-10.3:** The system shall provide a "Guided Suggestion" feature where the chatbot offers follow-up questions to help users explore their data further.
- **REQ-10.4:** The system shall allow users to export chat-generated insights directly into a PDF or a dashboard.

3.11 Predictive Analytics

3.11.1 Description and Priority

Priority: Low.

This section focuses on forward-looking insights. While the rest of the platform shows what *has* happened, Predictive Analytics uses historical patterns to estimate what *will* happen. This is essential for planning budgets, inventory, or sales targets.

3.11.2 Stimulus/Response Sequences

- The user selects a specific measure (e.g., 'Monthly Revenue') and toggles the "Forecast" switch.
- The system calculates the trend and extends the chart line into future months using a dotted line.
- The user adjusts a "What-if" variable (e.g., "What if marketing spend increases by 10%?").
- The system updates the predictive model to show the potential impact on future outcomes.

3.11.3 Functional Requirements

- **REQ-11.1:** The system shall include a Trend Forecasting engine that uses historical data points to project future values for any selected Measure.
- **REQ-11.2:** The system shall visually distinguish predicted data from actual data on charts using different line styles (e.g., dotted or dashed lines).
- **REQ-11.3:** The system shall provide a Confidence Score for every prediction to tell the user how likely the forecast is to be accurate.
- **REQ-11.4:** The system shall allow users to perform What-if Analysis, where they can change certain variables to see how it might affect the predicted future trend.

3.12 AI SQL Assistant

3.12.1 Description and Priority

Priority: Low.

This feature provides an intelligent side-panel for the SQL Query Interface. It is designed to assist technical users in writing complex queries faster and to help beginner analysts learn SQL by translating their thoughts into code. It acts as a bridge between natural language and structured database queries.

3.12.2 Stimulus/Response Sequences

- The user opens the SQL Editor and types a request into the NLP Side-Panel (e.g., "Find the top 10 products by profit margin").
- The AI Assistant generates the SQL code and displays it in the side-panel with an "Insert" button.
- The user highlights an existing SQL query and selects "Optimize" in the side-panel.
- The AI suggests a more efficient version of the code and explains the changes made.

3.12.3 Functional Requirements

- **REQ-12.1:** The system shall provide a dedicated NLP input window adjacent to the SQL editor for natural language instructions.
- **REQ-12.2:** The system shall feature Context-Aware Translation, using the specific metadata and schema of the uploaded dataset to ensure generated SQL is accurate.
- **REQ-12.3:** The system shall support Error Correction, where the AI analyzes SQL syntax errors and suggests the corrected code in the side-panel.

3.13 Other AI features

3.13.1 Description and Priority

Priority: Low.

This section covers supplemental AI features designed to improve data literacy and speed up decision-making. These tools assist users in the interpretation of results by providing plain-English summaries and identifying hidden patterns. These features are intended to lower the barrier to entry for non-technical users who may find complex visualizations difficult to analyze manually.

3.13.2 Functional Requirements

- **REQ-13.1: Chart Summarization** The system shall feature an AI-driven "Summarize" tool that generates plain-English takeaways from any active visualization, identifying key trends, peaks, and averages.
- **REQ-13.2: Automated Anomaly Explanation** The system shall use AI to detect "outliers" or unexpected spikes in data and provide a written hypothesis explaining which dimensions contributed most to that change.
- **REQ-13.3: Smart Visualization Recommendation** The system shall suggest the most appropriate chart type for a given query result (e.g., suggesting a time-series line chart for date-based data or a map for geographic data).
- **REQ-13.4: Natural Language Data Search** The system shall allow users to search for specific datasets or existing dashboards using conversational keywords instead of requiring exact filenames or paths.
- **REQ-13.5: Insight Pinning** The system shall allow users to save or "pin" an AI-generated summary directly onto a dashboard so that other collaborators can see the written explanation alongside the visual.

3.14 Version Control and History

3.14.1 Description and Priority

Priority: Low.

This feature acts as a "safety net" for the platform, allowing users to track every change made to their data inputs and dashboard layouts. Much like a Git-based workflow, it enables users to save specific versions of their work, compare changes over time, and revert to a previous state if an error occurs. This is particularly useful in collaborative environments where multiple people edit the same workspace.

3.14.2 Stimulus/Response Sequences

- A user makes significant changes to a dashboard layout and clicks "Commit Changes."
- The system saves the current state as a new version and prompts the user for a brief description of the changes.
- A user realizes they accidentally deleted a critical chart and opens the "Version History" panel.
- The system displays a chronological list of all saved versions (commits).
- The user selects a version from two days ago and clicks "Revert."
- The system restores the dashboard and the associated data inputs to exactly how they were on that date.

3.14.3 Functional Requirements

- **REQ-14.1:** The system shall track changes to data inputs, allowing users to see when a dataset was updated and providing the ability to "Pull" previous versions of the raw data.
- **REQ-14.2:** The system shall allow users to manually save a "Snapshot" of their entire dashboard, including chart configurations, filters, and layout positions.
- **REQ-14.3:** The system shall provide a visual "Diff" or log that highlights what has changed between two versions (e.g., "Chart A moved," "Filter B added").
- **REQ-14.4:** The system shall allow authorized users to restore any previous version of a dashboard or data mapping, overwriting current changes to fix errors.
- **REQ-14.5:** The system shall allow users to work on a "Draft" version of a dashboard without affecting the "Live" version seen by others, requiring a manual "Push" to make changes public.

4 Sprint 1 Scope: Data Ingestion & Validation

4.1 Sprint Goal

The objective of Sprint 1 is to implement the **Data Ingestion Wizard**. The system must support multi-format file uploads (CSV/Excel), allow users to define ingestion rules (Create/Replace/Append), and automatically quarantine corrupted data to ensure system integrity.

4.2 Deliverables

- **Ingestion Wizard UI**: A step-by-step React interface for file selection and rule definition.
- **Validation & Quarantine Engine**: A Node.js service that scans incoming data and redirects "broken" records to a separate storage bucket.
- **Streaming Foundation**: Initial setup for Socket.io or a similar protocol to handle live data feeds.
- **Flexible Schema Handler**: Backend logic to map incoming file headers to existing MongoDB collections based on user-selected rules.

4.3 Sprint 1 Work Distribution

Engineer 1 | Team Lead & Architect

- Define the logic for the "Ingestion Rules" (Create vs. Replace vs. Append) to ensure database atomicity.
- Architect the **Quarantine Workflow**: determine how "corrupted" data is flagged and stored without breaking the main pipeline.
- Standardize the JSON error objects for REQ-1.4 (Validation) to be sent back to the frontend.

Engineer 2 | Backend Developer

- Implement the main upload controller using **Multer** and **ExcelJS/Fast-CSV**.
- Develop the **Ingestion Wizard Logic**: write the backend functions that handle "Append" (insert) vs. "Replace" (delete then insert).
- Integrate **Socket.io** to lay the groundwork for REQ-1.3 (Real-time data feeds).

Engineer 3 | Backend Developer

- Implement secure file-handling middleware to prevent "Malicious File" uploads (security for REQ-1.1).
- Create protected API routes for the Ingestion Wizard using JWT.
- Ensure that only "Owners" of a dataset can perform a "Replace" or "Append" action.

Engineer 4 | Frontend Developer

- Build the **Step-by-Step Ingestion Wizard** (Step 1: Upload, Step 2: Choose Rule, Step 3: Mapping/Validation).
- Develop the **Quarantine Dashboard**: a view for users to see which files or rows failed validation (REQ-1.5).
- Implement Tailwind-based status trackers for real-time upload progress.

Engineer 5 | Database Engineer

- Setup the **Quarantine Collection** in MongoDB for storing corrupted/unverified data.
- Design the **Metadata Schema** to track the "Source" and "Ingestion Rule" used for every record.
- Configure MongoDB indexes to handle the "Replace" and "Append" operations efficiently to avoid performance lag.

Engineer 6 | QA & Documentation

- Create "Stress Test" files: intentionally corrupted CSVs and mismatched Excel files to verify the Quarantine logic (REQ-1.5).
- Document the "Ingestion Wizard" API flow in Postman.
- Validate that "Real-time" packets are correctly received and logged by the backend.

4.4 Technology Stack (Sprint 1)

4.4.1 Frontend

- **React.js & Tailwind CSS:** For the interactive step-by-step wizard.
- **Lucide-React:** For iconography (Upload icons, Quarantine/Warning symbols).

4.4.2 Backend

- **Node.js & Express:** Core API framework.
- **ExcelJS / Fast-CSV:** Libraries for parsing REQ-1.1 file types.
- **Socket.io:** To enable the REQ-1.3 real-time stream foundation.

4.4.3 Database & Storage

- **MongoDB & Mongoose:** For flexible metadata and data storage.
- **GridFS:** For temporary storage of files during the "Scan & Verify" phase (REQ-1.4).

4.5 Data Flow (Sprint 1)

1. **Trigger:** User initiates the "Upload Wizard" in the React UI.
2. **Upload & Scan:** File is sent to the Backend; the system immediately scans for format errors (REQ-1.4).
3. **Branching Logic:**
 - **Success:** File proceeds to "Step 2" where the user chooses the rule (Append/Replace/New).
 - **Failure:** Corrupted parts are shunted to the **Quarantine Folder** (REQ-1.5).
4. **Rule Execution:** The system modifies the MongoDB collection based on the user's choice (e.g., clearing the collection for a "Replace" rule).
5. **Persistence:** Final data is saved, and a "Success" notification is pushed to the UI.